

Chapter 55: Design YouTube

Personal Notes

YouTube looks simple: upload a video, then people watch it. Under the hood, it's a very different beast from any of the previous designs. The videos are HUGE (gigabytes each), they need to be re-encoded into a dozen different formats/resolutions to work on every device and network, they must stream smoothly to hundreds of millions of concurrent viewers across the globe, and every one of those viewers expects <2 second startup time. This is a MEDIA problem, not a data problem.

This chapter introduces a set of concepts you haven't seen yet: the encoding pipeline (batch processing at massive scale), adaptive bitrate streaming (HLS / DASH), and edge-heavy CDN architecture. Beyond YouTube, these patterns power Netflix, Twitch, Zoom, and every serious video platform. Master this design and you can tackle the whole video-streaming genre of interviews.

1. The Problem

Design a video-sharing service like YouTube. Users can UPLOAD videos, and other users can WATCH them. The system must efficiently deliver video to hundreds of millions of concurrent viewers on devices ranging from a \$50 phone on 3G to a 4K TV on gigabit fiber. It must also handle upload load, provide search, and support engagement features (likes, comments, subscriptions).

Upload path:

- Creator uploads raw video (say, 2 GB)
- System stores it
- Transcodes into 10+ formats (240p, 360p, 480p, 720p, 1080p, 4K)
× H.264, VP9, AV1 codecs
- Distributes to CDN edges worldwide
- Publishes; watchable

Watch path:

- Viewer clicks video
- Video player starts loading MANIFEST
- Manifest tells player about available resolutions
- Player picks best bitrate for its network
- Fetches 2-10 SECOND CHUNKS from CDN edge
- Adapts up/down as network changes
- Plays smoothly

2. Step 1 — Requirements Gathering

2.1 Functional Requirements

- Upload videos (up to several hours, up to ~10 GB raw)

- Watch videos at multiple resolutions with adaptive bitrate
- Like / dislike, comment, subscribe
- View counts, watch time analytics
- Search + recommendations
- (Optional) Live streaming, monetization, captions, download for offline

2.2 Non-Functional Requirements

- Scale — 2B users, 500 hours of video uploaded PER MINUTE, 5B daily views
- Playback latency — video starts within 2 seconds of clicking
- Playback smoothness — < 1% rebuffering events
- Availability — 99.99% for watch (uploads can queue if needed)
- Durability — uploaded videos NEVER lost

2.3 Constraints to Confirm

- Are we doing LIVE streaming or only VOD (video on demand)? — very different systems
- Max video length? — YouTube's is 12h; determines chunk count per video
- What quality tiers? — 240p → 4K? 8K?
- Do we serve globally, or just one region?

The critical clarification: "VOD vs Live?" is the single biggest split. VOD (uploaded, then watched later) can be preprocessed heavily — encode once, deliver cached. LIVE (streamed as it's happening) must transcode in real-time with minimal latency. Interviewers usually mean VOD unless they say otherwise; confirm this upfront.

3. Step 2 — Capacity Estimation

Uploads

500 hours of video / minute uploaded
= 8.3 hours / second

Average raw file: ~2 GB per hour of HD video
Peak upload throughput: ~17 GB/sec incoming raw video

Post-transcoding: 10 output variants × 500 hours/minute
= 5,000 encoded hours per minute of work to do

Encoding is CPU-heavy. Say 4× realtime per stream on a fast machine.
→ 5,000 / 4 = 1,250 CPU-hours per minute
→ Need ~75,000 CPUs continuously to keep up
(Peaks require even more — auto-scale by queue depth)

Watch Traffic

5B daily views × ~5 min average watch time = 25B minutes/day
= ~420,000 concurrent viewers on average
Peak (evening in major regions): × 10 = 4M concurrent viewers

Bandwidth per stream (1080p average): ~5 Mbps
Total peak egress: 4M × 5 Mbps = 20 Tbps globally

This is HUGE. It's more than most countries' internet capacity.
Which is why serving from CDN edge (not origin) is non-negotiable.

Storage

Encoded video storage per hour uploaded:
10 resolution/codecs variants × ~500 MB each
= ~5 GB stored PER hour of source

500 hours / minute × 60 min × 24 hr × 365 days = 260M hours/year
× 5 GB/hour = 1.3 EXABYTES per year of encoded video

Yes, exabytes. Most of it is cold – 90% of views go to the top 10% of videos. Cold storage tiers (Glacier, Coldline) are essential.

The bandwidth number is the takeaway: Uploading 500 hours/minute is bad. Storing exabytes is bad. But serving 20 Tbps globally is the truly hard part — no single data center can do this. That's why CDN edge caching is the DEFINING architectural feature of a video system. Everything else exists to serve the CDN.

4. Step 3 — API Design

```
Upload (multi-step, chunked):
  POST /api/upload/init                → returns upload_id +
presigned URL(s)
  PUT {presigned_url}                 → client uploads chunks
directly to S3
  POST /api/upload/complete/:upload_id → notifies server; encoding
begins
  GET /api/upload/status/:upload_id    → poll for encoding progress

Metadata:
  POST /api/videos                     → create video (title,
description, tags)
  GET /api/videos/:video_id            → metadata for playback

Playback:
  GET /api/videos/:video_id/manifest   → returns HLS/DASH manifest
URL
```

```
GET {cdn_edge}/videos/:id/manifest.m3u8    → the manifest itself
(served from CDN)
GET {cdn_edge}/videos/:id/720p/segment_042.ts → video chunk (served
from CDN)
```

Engagement:

```
POST /api/videos/:video_id/like
POST /api/videos/:video_id/comment
POST /api/videos/:video_id/view          → log view event (async)
```

Direct-to-S3 upload is critical: The client uploads directly to S3 via presigned URLs — bypassing your app servers entirely. Otherwise a 2 GB upload would tie up an app server for minutes, consuming memory and bandwidth. Presigned URLs are how every serious upload service handles large files.

5. Step 4 — Data Model

```
Table: videos                                     ← Metadata
(MySQL/PostgreSQL, sharded)
  video_id (PK) | uploader_id | title | description | duration
                  | status ("processing" | "ready" | "failed")
                  | uploaded_at | manifest_url | thumbnail_url

Table: video_renditions                           ← One row per encoded
variant
  video_id | resolution | codec | bitrate_kbps | segment_dir_url

Table: view_events (Kafka → Cassandra/BigQuery)   ← Watch analytics
  video_id | user_id | timestamp | watch_seconds | resolution

Table: comments (sharded by video_id)
  video_id | comment_id | user_id | text | created_at |
parent_comment_id

Table: likes (sharded by video_id)
  video_id | user_id | created_at

Blob store (S3 / GCS):
  raw/{video_id}.mp4                                ← original upload
  encoded/{video_id}/{resolution}/{codec}/manifest.m3u8
  encoded/{video_id}/{resolution}/{codec}/segment_001.ts, 002.ts, ...
  thumbnails/{video_id}/{time}.jpg
```

6. The Core Concept — Video Encoding

Every uploaded video must be RE-ENCODED into multiple resolutions and codecs before it can be served. This is the workhorse of the system — encoding is CPU-intensive, embarrassingly parallel, and produces most of your storage bill.

Why Re-encode?

- The uploaded file might be 4K H.265 — a phone on 3G can't handle it
- Different devices support different CODECS (H.264 works everywhere; VP9 and AV1 are more efficient but less universal)
- Users on slow networks need LOWER RESOLUTIONS at lower bitrates
- Adaptive bitrate streaming REQUIRES multiple pre-encoded resolutions

Codecs vs Resolutions

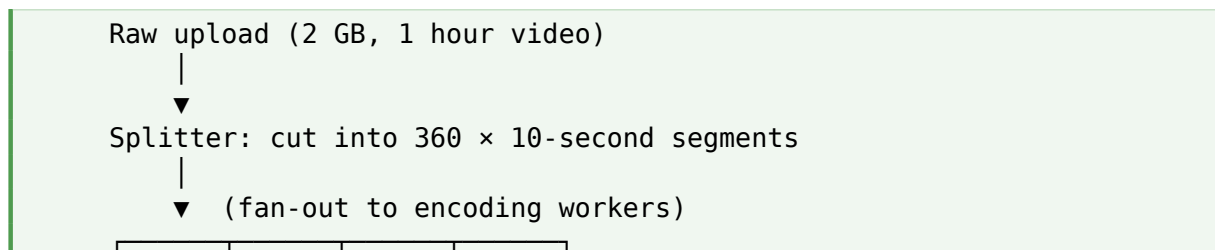
Codec	Description	Where it's used
H.264 (AVC)	Universal (works on every device made in the last 15 years)	Legacy fallback, mobile 3G/4G
H.265 (HEVC)	Better compression than H.264, but patent-encumbered	iOS/Safari; some Android
VP9	Google's royalty-free alternative to H.265	Chrome, YouTube's primary codec
AV1	Newest, most efficient, royalty-free	Modern devices; still expanding

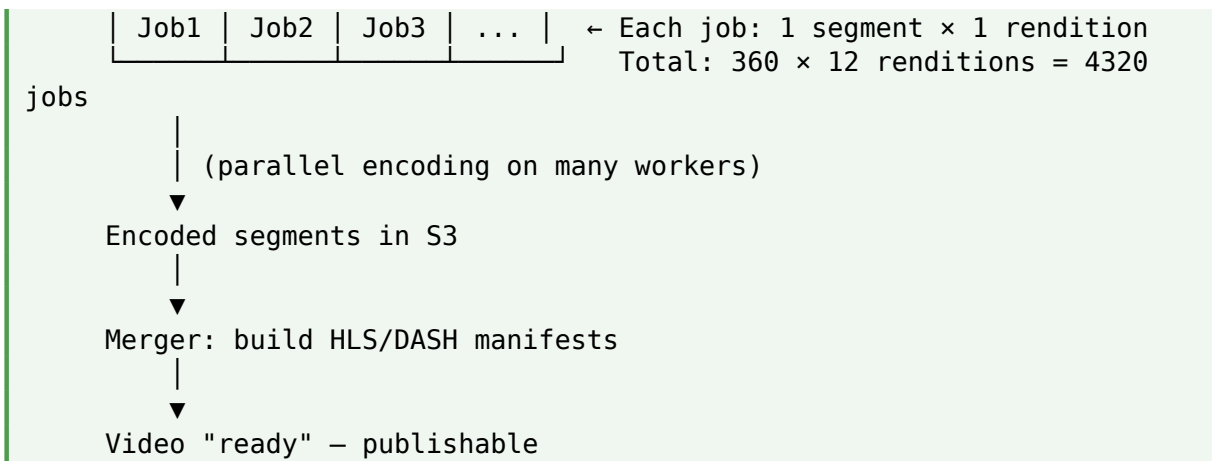
Typical rendition matrix: 6 resolutions (240p, 360p, 480p, 720p, 1080p, 4K) × 2-3 codecs (H.264 + VP9 + optionally AV1) = ~12-18 output files per upload.

7. The Encoding Pipeline

Encoding is a BATCH JOB, not a request-response operation. When a user uploads, the workflow is:

1. Client uploads raw video to S3 (bypassing app servers)
2. Server receives "upload complete" event
3. Message pushed to encoding queue (Kafka topic)
4. SPLIT the video into segments (e.g., 10-second chunks)
5. FAN-OUT: each segment × each rendition = one encoding job
6. Encoding workers (running FFmpeg) pull jobs, encode, upload output to S3
7. MERGE: reassemble encoded segments into full streams + manifests
8. Video marked "ready", metadata updated, CDN warmed with popular renditions





Why split then merge: Encoding a 1-hour video sequentially on one machine could take an hour. Split into 10-second chunks, encode all in parallel across many workers, and you finish in ~10 seconds (limited by the slowest chunk). This is a classic MAPREDUCE / FORK-JOIN pattern applied to video encoding. Netflix and YouTube both use it.

8. Adaptive Bitrate Streaming — HLS & DASH

The player doesn't download a 1 GB video file. It downloads a MANIFEST that lists many small chunks at multiple quality levels, then decides which chunks to fetch based on network conditions. This is ADAPTIVE BITRATE STREAMING (ABR) — the video quality "adapts" to your network in real time.

How ABR Works

Manifest (HLS example, .m3u8):

```
# Available quality levels
1080p (5 Mbps) → segments in /1080p/
720p (2.5 Mbps) → segments in /720p/
480p (1 Mbps) → segments in /480p/
360p (500 Kbps) → segments in /360p/
240p (250 Kbps) → segments in /240p/
```

Player logic:

- Start with mid-tier (720p) to be safe
- Measure download time of first chunk
- If chunk arrived way before playback needed → try HIGHER quality
- If chunk barely arrived in time → drop to LOWER quality
- If chunk missed the deadline → SEVERE drop; rebuffer briefly

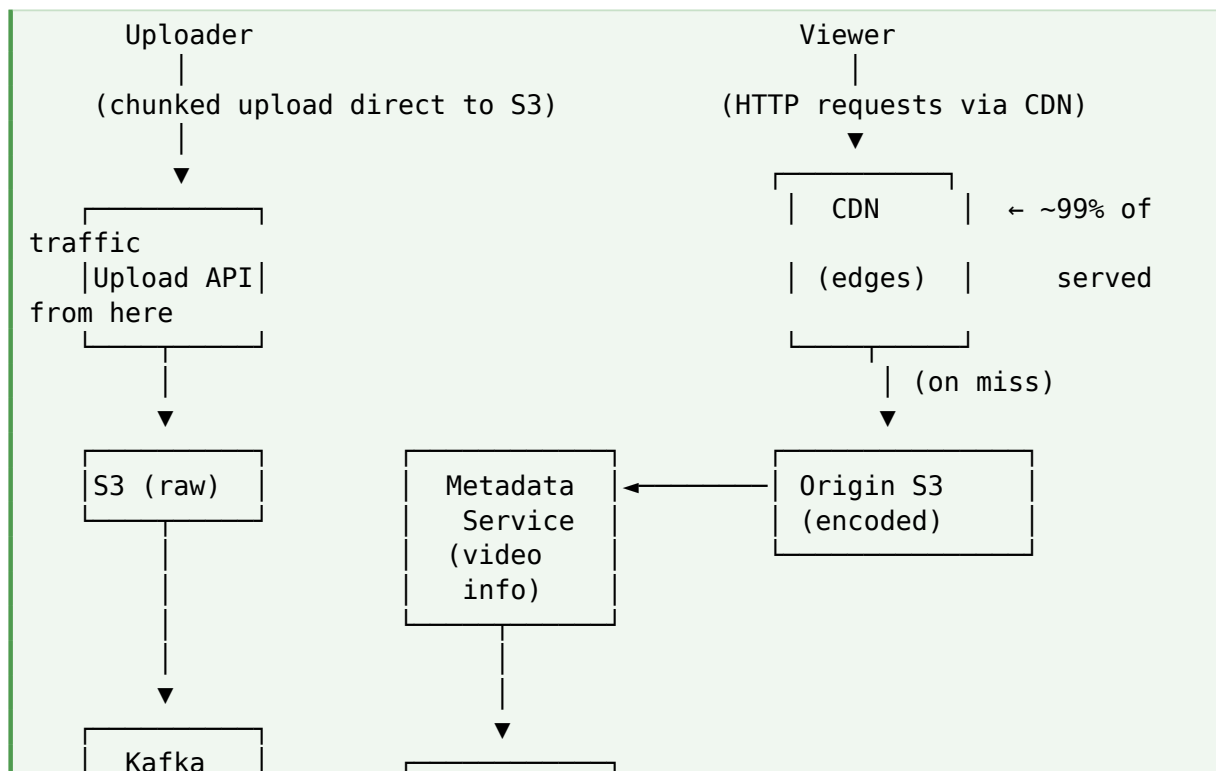
Chunks are ~2-10 seconds. Switching between qualities is seamless.

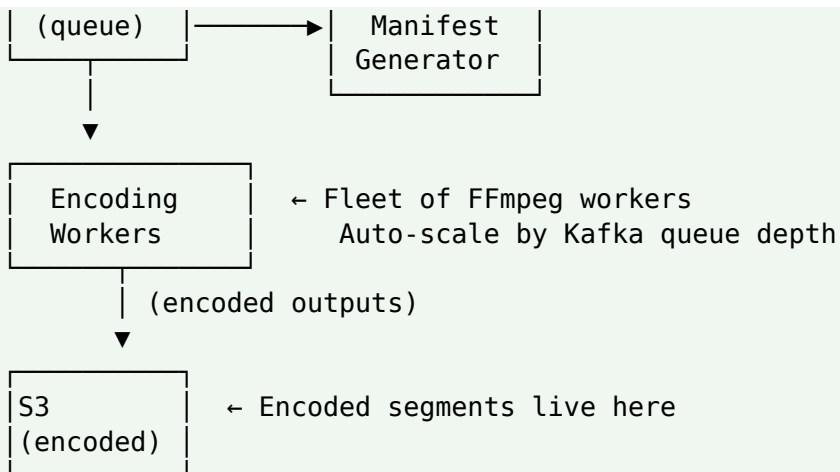
HLS vs DASH

Aspect	HLS	DASH
Full name	HTTP Live Streaming	Dynamic Adaptive Streaming over HTTP
Origin	Apple	MPEG standard (open)
Manifest	.m3u8 (text file)	.mpd (XML file)
Segment format	.ts (MPEG-TS)	.mp4 (fragmented MP4)
Codec support	H.264, H.265, HEVC	Any codec — codec-agnostic
iOS/Safari	Native support	Requires JS player
Industry use	YouTube fallback, Apple ecosystem	Netflix primary, Android, most Western streaming

For interviews: Say "we serve HLS and DASH manifests based on the client's platform." iOS clients get HLS (Apple requires it for App Store apps). Everything else gets DASH. The chunks are usually shared — same MP4 fragments referenced by both manifest types.

9. Step 5 — High-Level Architecture





Metadata: MySQL (sharded) for video info
Comments/likes/subs: separate services
View events: Kafka → BigQuery for analytics
Search & recommendations: separate ML-driven services

10. Deep Dive: The CDN Architecture

The CDN is THE most important piece of a video service. 99% of watch bandwidth is served from CDN edges — never touching your origin. If it were served from origin instead, the bandwidth bill alone would be catastrophic (imagine paying for 20 Tbps of cross-country transfer).

CDN Tiers

- EDGE (POP — Point of Presence) — one in every major city, holds recently-watched videos in the region
- REGIONAL — larger caches per continent; edge misses hit here first
- ORIGIN — the master copy in S3, only hit if regional cache misses too

Viewer in Mumbai clicks a viral cricket highlight

1. Browser resolves `cdn.example.com` → Mumbai POP IP
2. Requests `/videos/xyz/720p/segment_042.ts`
3. Mumbai POP: HIT? → serve immediately (5ms latency)
| MISS?
▼
4. Regional cache (Singapore): HIT? → serve, populate Mumbai cache
| MISS?
▼
5. Origin (US-East S3): serve, populate Singapore + Mumbai
(100+ ms latency, expensive)

Result for viral videos: hit rate 95%+ at edge → super fast globally.

Cache Warming for Predictable Traffic

Netflix knows that "Squid Game S02E01" will be popular. When they publish, they PRE-POPULATE the CDN edges before the release. YouTube does similar for scheduled premieres. Cold-start delays are avoided by pushing content to edges proactively.

The long-tail problem: 10% of your videos get 90% of the views (Zipf distribution) — those are easy to cache. But that other 90% of videos still gets watched sometimes, and each miss to origin is expensive. YouTube's approach: keep the head content on edge, mid-tier content on regional, cold tail from origin. Accept that cold videos have slower startup.

11. Deep Dive: The Upload Flow

1. Client: POST /api/upload/init
with metadata (title, size, chunk_size)
Server: creates upload_id, returns presigned URLs for chunks
2. Client: uploads chunks directly to S3 in parallel (5-10 at a time)
If chunk fails: RETRY that chunk only — doesn't resend the whole file
3. Client: POST /api/upload/complete/{upload_id}
Server: verifies all chunks arrived, marks upload complete
publishes "encode_video" event to Kafka
4. Encoding pipeline:
 - Splitter service: cut into 10-sec segments
 - For each segment × rendition: enqueue an encoding job
 - Workers (FFmpeg on GPU or CPU) process jobs in parallel
 - Merger service: assemble segments, generate HLS/DASH manifests
 - Publish to S3, mark video "ready"
5. Client: polls /api/upload/status
When status = "ready" → video visible on platform

Resumable uploads are critical: Users uploading 2 GB videos on flaky connections shouldn't have to restart from scratch on failure. Chunked uploads with a manifest of "which chunks are done" allow resumption. Cloud providers offer this natively (S3 multipart upload, Google Cloud Storage resumable uploads).

12. Deep Dive: The Watch Flow

1. User clicks video_id "xyz"
2. Client: GET /api/videos/xyz → returns metadata + manifest URL
3. Client: GET {cdn}/videos/xyz/manifest.m3u8
Manifest lists all quality tiers + segment URLs

4. Player selects starting quality (usually 720p or auto-detect)
5. Player: GET {cdn}/videos/xyz/720p/segment_001.ts
GET .../segment_002.ts
... (keeps buffer of 2-3 segments ahead)
6. Player measures throughput:
 - Fast? → switch to 1080p on next segment
 - Slow? → drop to 480p on next segment
7. Client: POST /api/videos/xyz/view (async, fire-and-forget)
Server: buffers view event to Kafka → BigQuery for analytics

Startup Optimization

- PRE-FETCH first segment as soon as user hovers video (before click)
- START LOW — begin with 480p, quickly ramp up if bandwidth allows
- SHORT initial chunks — first chunk is ~2 sec, later ones ~10 sec
- Use HTTP/2 to parallelize manifest + first-chunk requests

13. Deep Dive: Metadata Service

The metadata service is the small-data workhorse. Every video has metadata (title, description, uploader, thumbnails, duration, tags), and every video load includes a metadata fetch. Compared to video bytes, metadata is tiny — but it's fetched on every view.

- Store in SHARDED SQL (MySQL/PostgreSQL) with video_id as shard key
- Cache heavily in Redis — hit rate for popular videos approaches 100%
- Update metadata is rare (edits by creator) — write path is not hot
- For view counts, use a SEPARATE fast counter (see next section)

14. Deep Dive: View Counts & Analytics

"1.2M views" is displayed on every video. Incrementing this counter synchronously on every view would kill a database — 5B/day = ~60K writes/sec baseline, with viral spikes reaching millions per second.

Approach

9. Client sends view event to /api/view — server writes to KAFKA (fire-and-forget)
10. Kafka consumer aggregates counts in batches (say, every 30 seconds)
11. Aggregated counts written to Cassandra or a fast key-value store
12. For deep analytics (watch time, retention curves), pipeline into BigQuery
13. Display view counts from a REDIS CACHE, updated periodically

View count deduplication: A user refreshing a video shouldn't add 100 views. Dedupe by (user_id, video_id) within a time window, or require > 30 sec watch time to count. YouTube's actual view count algorithm is famously secret and adversarial-resistant — but the interview-level answer is "async aggregation with dedup."

15. Deep Dive: Search & Recommendations

Both are ML-heavy services that sit OUTSIDE the video pipeline. Discuss them briefly — deep coverage is out of scope for this design.

Search

- Index video metadata (title, description, tags, transcripts) in Elasticsearch
- Rank by (relevance to query, popularity, freshness, personalization)
- Auto-suggestions from a trie of past queries (see Ch 36 tries!)

Recommendations

- For each user, generate a ranked list of "videos you might like"
- Two-stage: CANDIDATE generation (thousands of videos) → RANKING (ML model)
- Precompute per-user recommendations offline every few hours; refresh with recent signals
- YouTube's system is famously complex, but the interview answer is "two-stage ranker with async precompute + fresh signals at serve time"

16. Additional Features

16.1 Comments

Standard commenting system — comments table sharded by video_id, threaded via parent_comment_id. High-traffic videos get millions of comments; paginate on read. Reuse patterns from the news feed and chat chapters.

16.2 Likes / Dislikes

Async counter increments (same pattern as view counts). Per-user like state stored in a likes table sharded by video_id. Display denormalized total in metadata.

16.3 Subscriptions

Users subscribe to channels. Same pattern as follows in the news feed chapter. New uploads notify subscribers — potentially fan-out on write for small channels, on-read for celebrities (channels with millions of subs).

16.4 Live Streaming

Different beast — real-time encoding, sub-5-second latency, no re-encoding pipeline. Uses RTMP or WebRTC ingest, low-latency HLS or WebRTC for playback. Would deserve a chapter of its own.

16.5 Monetization / Ads

Ads inserted client-side (VAST/VMAP standards). Server-side ad insertion (SSAI) stitches ads into the video manifest for seamless playback. Separate ad service handles targeting + delivery.

17. Failure Modes

Failure	Mitigation
Upload fails mid-way	Chunked upload — resume from last successful chunk; no full restart
Encoding worker dies	Job goes back to Kafka queue; another worker picks it up (at-least-once)
CDN edge cache goes cold	Miss falls through to regional / origin — slower but still works
Origin S3 region outage	Multi-region replication; failover to secondary region
Encoding backlog grows	Auto-scale workers by queue depth; delay non-priority uploads
Manifest generator down	Video shows as "processing" longer; already-published videos unaffected
View analytics backlog	Displayed counts lag by minutes; no user impact beyond staleness
Live viral event (Super Bowl)	Pre-warm CDN + throttle upload; scale watch capacity ahead of time

18. Trade-offs Summary

Decision	Trade-off
Encode all resolutions vs on-demand	Pre-encode all = fast serve, more storage; on-demand = save storage, slower first-view
HLS vs DASH	Serve both — HLS for iOS, DASH for everything else
Chunk length (2s vs 10s)	Short = fast start + agile adaptation; long = smaller manifests + less overhead
More codecs (VP9, AV1) vs fewer	More = better compression on modern devices; fewer = simpler pipeline
Warm CDN proactively vs reactively	Proactive = higher hit rate for launches; reactive = simpler + cheaper

Decision	Trade-off
Sync view counter vs async	Sync = accurate real-time; async (Kafka) = scales to viral spikes
Multi-region storage vs single	Multi = HA + locality; single = simpler + cheaper

19. Common Mistakes

Mistake 1: Serving video from origin

At 20 Tbps peak egress, serving video from your own data centers would cost more than most companies' entire revenue. CDN edge is non-negotiable. If you skip this, the interviewer will steer you here.

Mistake 2: Uploading through app servers

Streaming 2 GB uploads through app servers ties them up and consumes bandwidth twice (client → server → S3). Use presigned S3 URLs so client uploads directly.

Mistake 3: Encoding sequentially

A naive design encodes the whole video on one machine — an hour of video takes an hour. Split into chunks, encode in parallel across many workers. Netflix and YouTube both do this.

Mistake 4: Not mentioning adaptive bitrate

Serving one video file at one resolution means: users on 3G buffer forever, users on gigabit fiber get 480p garbage. ABR (HLS/DASH) is table stakes for any modern video service.

Mistake 5: Sync-updating view counts

Every hot video getting synchronous view count updates would DDoS your database. Async aggregation via Kafka is the standard pattern.

Mistake 6: Ignoring the long-tail storage cost

1.3 exabytes/year is a lot. Cold storage tiers (Glacier, Coldline) for videos with < 1000 views. Delete/downsample very old videos with no views. YouTube probably has dedicated systems just for this.

Mistake 7: Treating live streaming like VOD

If the interviewer asks about live, DON'T describe the encoding pipeline above — that's for pre-recorded video. Live needs real-time transcoding, chunked ingest (RTMP/WebRTC), low-latency HLS. Explicitly clarify VOD vs live upfront.

20. Best Practices

- Client uploads directly to S3 via presigned URLs — never through app servers
- Split video into chunks, encode in parallel — fork-join pattern
- Serve HLS to iOS/Safari, DASH to everything else
- Store multiple renditions (240p → 4K, multiple codecs) — pre-encoded
- Serve 99%+ of bytes from CDN edge — never origin at scale
- Pre-warm CDN for predictable spikes (premieres, viral events)
- Async view counter aggregation via Kafka + batch increments
- Multi-tier storage — hot in fast storage, cold in Glacier/Coldline
- Resumable uploads via chunked S3 multipart
- For large channels, fan-out subscribers like a news feed (Ch 52)

21. Quick Reference

Concept	Meaning
Video encoding	Transcoding raw video into multiple formats/resolutions
Codec	Compression algorithm (H.264, VP9, AV1, etc.)
Resolution / Bitrate	Video dimensions + data rate; multiple per video
Rendition	One specific (resolution, codec) combination
HLS	Apple's adaptive streaming — .m3u8 manifest, .ts chunks
DASH	MPEG's adaptive streaming — .mpd manifest, .mp4 chunks
Adaptive Bitrate (ABR)	Player switches quality tiers based on network conditions
Manifest	Text/XML file listing available renditions + segment URLs
Chunk / Segment	2-10 second video slice; unit of transfer
Presigned URL	Temporary S3 URL for direct client-to-storage upload
CDN edge / POP	Geographic cache serving 99%+ of bytes
Fork-join encoding	Split video into chunks, encode in parallel, merge
Zipf distribution	Watch popularity — 10% of videos get 90% of views

22. Related Interview Problems

The video-streaming patterns from YouTube apply to many other systems:

14. Design Netflix — VOD focused; pre-encoded; heavy pre-fetch to CDN
15. Design Twitch — LIVE streaming; low latency; real-time chat + streams
16. Design TikTok — short-form vertical video, ranked feed (Ch 52), fast startup
17. Design Zoom / Google Meet — real-time WebRTC, no encoding pipeline
18. Design Spotify / Apple Music — audio streaming (much easier — smaller files, single stream)
19. Design a Podcast Platform — audio + episodes + subscriptions
20. Design a Photo Sharing Service (Instagram) — images instead of video, similar CDN patterns
21. Design an Online Video Editor — media pipeline + real-time preview

23. Final Takeaway

YouTube's design is defined by the FOUR unique concepts you didn't need in previous chapters: (1) the encoding pipeline as a batch fork-join workflow, (2) adaptive bitrate streaming for smooth playback across network conditions, (3) an edge-heavy CDN architecture serving 99%+ of bytes, and (4) async aggregation for view counts that scale to viral spikes. If you can articulate all four, you've cracked the video-streaming genre — Netflix, Twitch, TikTok are variations.

For interviews, memorize the flow: chunked upload direct to S3 → encoding pipeline (split/fan-out/encode/merge) → multiple pre-encoded renditions in S3 → CDN with edge/regional/origin tiers → HLS+DASH manifests → adaptive bitrate playback → async view counters via Kafka. Then be ready for the deep-dive questions: What if a chunk fails to encode? How do you handle a viral video? How do you serve globally?

Key Takeaway: YouTube = media pipeline problem. Client uploads DIRECT to S3 via PRESIGNED URLs. ENCODING pipeline splits video into chunks, encodes each × (resolution × codec) in parallel via FFmpeg workers, merges into manifests. Store MULTIPLE RENDITIONS (240p → 4K, H.264/VP9/AV1). ABR streaming via HLS (iOS) + DASH (others) — client adapts quality based on network. Serve 99%+ from CDN EDGE, never origin. View counts async via Kafka. Cold storage for the long tail. Pre-warm CDN for known spikes.

24. What's Next?

Five canonical designs done (URL shortener, feed, chat, Uber, YouTube). The core patterns are covered — now branch into more specialized designs:

- Design a Rate Limiter — token bucket at distributed scale, focused algorithmic depth
- Design a Distributed Cache — Redis internals, consistent hashing, replication
- Design a Notification System — cross-channel delivery, priorities, deduplication
- Design a Search Autocomplete — trie-based, prefix caching, popularity scoring

- Design a Payment System — idempotency, sagas, PCI compliance
- Design a Web Crawler — distributed fetching, politeness, dedup
- Deep Dive: Databases Internals — B-trees, LSM trees, WAL, MVCC
- Deep Dive: Kafka Architecture — partitions, consumer groups, exactly-once semantics
- Deep Dive: Consensus Algorithms — Raft, Paxos, leader election